

Lead & Tender Detection Engine

Phase 1 — Scraping & Relevance Scoring Module

CSTAM 3.0 Challenge — OliveSoft Automated RFP Intelligence System

Component	Public tender discovery, parsing, and relevance classification
Primary source	HAICOP / <code>marchespublics.gov.tn</code> (official Tunisian public procurement portal)
Secondary source	WebManagerCenter (private/parapublic tender news, disabled by default)
Output	<code>detection_queue.jsonl</code> — structured, deduplicated, scored leads

1 Overview

This module implements Phase 1 of the CSTAM 3.0 challenge: automated detection of public and private IT tenders relevant to OliveSoft’s expertise (custom software engineering, data/Bi integration, and CRM/ERP consulting). It continuously scans external sources, extracts structured tender data, scores each item for software/domain relevance, deduplicates against previously seen records, and exports the qualifying leads to a JSON Lines queue for downstream phases (RAG-based prospect research and proposal generation).

The module is self-contained, dependency-light (`requests` and `beautifulsoup4`), and designed to run as a standalone script or a scheduled job (e.g. inside an n8n workflow or a Colab/cron cell).

2 Architecture

2.1 Data sources

- **HAICOP** (*primary, enabled by default*) — Tunisia’s official public procurement portal. Chosen as the default source because it is open government data with no reproduction restriction, unlike `appeloffres.com` (ToS forbids content reuse) or TUNEPS (`robots.txt` disallows automated access). It also exposes a structured “*Type de commande*” field, a far more reliable relevance signal than free-text keyword matching.
- **WebManagerCenter** (*secondary, disabled by default*) — a Tunisian business-news outlet that occasionally covers private and parapublic tenders never published on HAICOP. Kept implemented for future use but requires closer monitoring (see §8).

Both sources are toggled independently via `CONFIG["enable_haicop"]` and `CONFIG["enable_wmc"]`.

2.2 Pipeline

Each source follows the same four-stage pipeline:

1. **Discovery** — locate candidate URLs or IDs (HAICOP: recent tender IDs from the homepage widget; WMC: archive listing pages and keyword-search result pages).
2. **Fetch & parse** — download each page (with retry/back-off) and extract structured fields from the article body, not from page furniture (navigation, tag clouds, related-article widgets).
3. **Relevance classification** — score the tender against OliveSoft’s domain (§3).
4. **Deduplication & persistence** — upsert into SQLite keyed on a content hash; only genuinely new, still-fresh, relevant tenders are exported.

3 Relevance Classification

Relevance is intentionally a conjunction of two independent checks, not a single keyword hit:

Source	Scoring logic
HAICOP	Base score from the official <i>Type de commande</i> category (rejecting hardware-only categories such as toners/consumables); bonus score if the title matches a software keyword; further bonus if the title matches OliveSoft’s specific expertise stack (BI, CRM, ERP, Salesforce, MuleSoft, Power BI, Qlik/Talend, data warehousing).
WebManagerCenter	A text must <i>first</i> contain a genuine tender-signal phrase (e.g. “ <i>avis d’appel d’offres</i> ”, “ <i>cahier des charges</i> ”, “ <i>soumissionnaire</i> ”) — this filters out news articles and opinion pieces that merely discuss IT or tenders in general. Only if that check passes is the text scored against the software-keyword list.

Over-generic single words (*développement*, *système*, *data*, *plateforme*, ...) are deliberately excluded as stand-alone triggers and only counted as part of specific compound phrases (“*système d’information*”, “*développement web*”, ...), to avoid false positives from unrelated company names or page boilerplate. A configurable threshold (`export_relevance_threshold`, default 0.3) determines whether a scored tender is exported.

4 Data Model

Every detected tender is normalized into a single **Tender** record:

Field	Type	Description
<code>source</code>	str	"HAICOP" or "WebManagerCenter"
<code>external_id</code>	str	Source-native identifier (tender ID or URL slug)
<code>title</code>	str	Tender object / article headline
<code>organization</code>	str	Issuing/publishing entity
<code>domain_type</code>	str	Official category (HAICOP) or "Annonce presse" (WMC)
<code>published_date / deadline</code>	str	DD-MM-YYYY, validated before storage
<code>budget</code>	float	Parsed guarantee/bond amount, when present
<code>is_it_relevant</code>	bool	Result of the relevance classifier
<code>confidence_score</code>	float	0–1 relevance score
<code>relevance_reasons</code>	str	Human-readable audit trail of matched signals

4.1 Title resolution

The tender title is read from the page’s *Objet* field, with a fallback to *Descriptif* when *Objet* is shorter than `min_title_length` (20 characters). This was added after live testing surfaced tenders (e.g. Tender-96634) where *Objet* holds a short internal reference code ("tma systeme AMEN") while the actual, actionable description lives in *Descriptif*.

4.2 Deduplication

Each record is keyed by an MD5 hash: HAICOP uses `title + organization + deadline + source`; WebManagerCenter uses `title + source + publication date`, since free-text organization extraction on news copy is a heuristic and less stable across republished or duplicate posts. A cheap pre-fetch check on (`source`, `external_id`) also skips re-downloading HAICOP pages already stored, since a published tender page’s content is immutable.

4.3 Storage

Two SQLite tables back the pipeline:

- **tenders** — one row per unique tender (full schema above, plus `status`, `first_seen`, `last_seen`).
- **events** — append-only audit log (`tender_hash`, `event_type`, `timestamp`) for traceability.

5 Output Data Structure

The pipeline produces the same logical record in two parallel forms: a line-delimited JSON feed for downstream consumers, and a relational mirror in SQLite for querying and audit. Both are generated from the same **Tender** dataclass, so the two never drift out of sync.

5.1 JSON Lines export (`detection_queue.jsonl`)

- **Format:** JSON Lines — one self-contained JSON object per line, UTF-8 encoded, no enclosing array and no trailing comma.
- **Write mode:** append-only ("a"); the file is never rewritten, so it is safe to tail or stream and safe to re-run the scraper without corrupting earlier exports.
- **Filtering:** only records with `is_it_relevant = true` are written here; every discovered record (relevant or not) is still persisted in SQLite for audit.
- **Field set:** every line contains exactly the fourteen fields of the **Tender** dataclass (`dataclasses.asdict`), listed below with type and origin.

Key	JSON type	Notes
<code>source</code>	string	"HAICOP" or "WebManagerCenter"
<code>external_id</code>	string	Tender ID (HAICOP) or URL slug (WMC)
<code>title</code>	string	Tender object / article headline
<code>organization</code>	string	Issuing buyer or press-detected entity; "" if not found
<code>domain_type</code>	string	Official category (HAICOP) or fixed "Annonce presse" (WMC)
<code>published_date</code>	string	"DD-MM-YYYY", or "" if absent/unparseable
<code>deadline</code>	string	"DD-MM-YYYY", or "" (WMC never sets this)
<code>budget</code>	number null	Provisional guarantee amount in TND; null if not stated
<code>region</code>	string	Execution region (HAICOP only; "" for WMC)
<code>url</code>	string	Canonical source URL for the record
<code>raw_excerpt</code>	string	First 500 characters of the parsed detail block, kept for manual review
<code>is_it_relevant</code>	boolean	Output of the relevance classifier (§3)
<code>confidence_score</code>	number	0.0–1.0 relevance score
<code>relevance_reasons</code>	string	Semicolon/comma-separated audit trail of matched signals

5.1.1 Example — HAICOP record

```
{
  "source": "HAICOP",
  "external_id": "Tender-48211",
  "title": "Mise en place d'une plateforme de business intelligence",
  "organization": "Ministere des Finances",
  "domain_type": "Informatique",
  "published_date": "02-10-2026",
  "deadline": "14-11-2026",
  "budget": 5000.0,
  "region": "Tunis",
```

```

"url": "https://www.marchespublics.gov.tn/fr/appels-doffres/Tender-48211",
"raw_excerpt": "Numero A.0: 2026/12 ... Objet: Mise en place d'une ...",
"is_it_relevant": true,
"confidence_score": 0.8,
"relevance_reasons": "category:Informatique; title:business intelligence; olivesoft-expertise:business intelligence"
}

```

5.1.2 Example — WebManagerCenter record

```

{
  "source": "WebManagerCenter",
  "external_id": "stb-lance-un-appel-doffres-pour-sa-refonte-si",
  "title": "La STB lance un appel d'offres pour la refonte de son SI",
  "organization": "La STB",
  "domain_type": "Annonce presse",
  "published_date": "18-09-2026",
  "deadline": "",
  "budget": null,
  "region": "",
  "url": "https://www.webmanagercenter.com/2026/09/18/stb-lance-un-appel-doffres-pour-sa-refonte-si/",
  "raw_excerpt": "La Societe Tunisienne de Banque (STB) a annonce le lancement ...",
  "is_it_relevant": true,
  "confidence_score": 0.4,
  "relevance_reasons": "systeme d'information,informatique"
}

```

Note that `deadline`, `budget` and `region` are structurally empty/null for WebManagerCenter records: press articles do not carry a formal submission deadline, guarantee amount, or execution region, so these fields are reserved for HAICOP's structured detail page.

5.2 SQLite mirror (`ctam_tenders.db`)

The `tenders` table stores the identical fourteen fields as columns, plus four pipeline-management columns not present in the JSON export:

Column	Type	Purpose
<code>hash</code>	TEXT (PK)	MD5 dedup key (§4.2)
<code>status</code>	TEXT	Lifecycle marker, currently always "new" on insert
<code>first_seen</code>	TEXT (ISO 8601)	UTC timestamp of first detection
<code>last_seen</code>	TEXT (ISO 8601)	UTC timestamp updated on every re-encounter

A companion `events` table logs one "discovered" row per new tender (`tender_hash`, `event_type`, `timestamp`), giving a simple append-only audit trail independent of the mutable `tenders` table. Unlike the JSONL export, SQLite holds *every* detected record — relevant or not, fresh or stale — so it is the authoritative source for statistics (`TenderStore.stats()`) and for re-deriving the export if the relevance threshold changes later.

6 Configuration Reference

Key parameters exposed in `CONFIG`:

Key	Purpose
<code>enable_haicop</code> / <code>enable_wmc</code>	Source on/off toggles
<code>haicop_id_backfill</code>	Number of tender IDs scanned backward per run
<code>wmc_pages_to_scan</code> / <code>wmc_search_keywords</code>	WMC discovery breadth
<code>wmc_max_age_days</code>	Freshness cutoff for export (older items kept for audit only)
<code>export_relevance_threshold</code>	Minimum score required to export a tender
<code>request_timeout</code> / <code>request_delay_seconds</code> / <code>max_retries</code>	Network politeness & resilience
<code>db_path</code> / <code>jsonl_path</code>	Output locations

7 Usage

```
pip install requests beautifulsoup4
python olivesoft_rfp_scraper.py
```

Each run appends newly discovered, relevant tenders to `detection_queue.jsonl` and persists the full detection history (relevant or not) to `ctam_tenders.db`. The exact structure of both outputs, with full field-by-field examples, is documented in §5.

8 Known Limitations & Next Steps

- **Multi-lot tenders under-report budget/region.** Verified live against Tender-103899 (2 lots, guarantees 700 and 7000 TND): since field extraction is a single whole-text label search, only the *first* lot's **region** and **budget** are ever captured, and on some page layouts the region value can also trail into the neighbouring budget label's text (e.g. "ZAGOUANE Montant du cautionnement provisoire : 700" instead of just "ZAGOUANE"). A proper fix means parsing each LotN block as its own structure rather than one whole-page label search — left as a follow-up.
- **TUNEPS** is intentionally excluded (`robots.txt` disallow, prior HTTP 403 responses); revisit only via an official API/ partnership channel.
- **WebManagerCenter** remains off by default: its article structure is theme-dependent and its search index mixes recent notices with decade-old archives, requiring the freshness filter and closer monitoring before re-enabling in production.
- Relevance scoring is keyword/category-based; a natural next step (Phase 2 of the challenge) is to route qualifying tenders into the RAG layer for deeper semantic matching against OliveSoft's project history.
- No scheduling/orchestration is included here by design — this module is meant to be invoked by the n8n workflow layer.